

MSA Day 2 실습 - 맛집 리뷰 서비스 확장

Chapter 1: 환경 정리 & 사전 준비

1-1. 로컬 환경 정리

오전 강의에서 Docker Compose로 띄운 컨테이너, Day 1 실습에서 IntelliJ로 실행한 서비스가 남아있을 수 있다. 포트 충돌을 방지하기 위해 먼저 정리한다.

IntelliJ에서 실행 중인 서비스 중지

IntelliJ에서 Run 중인 서비스가 있으면 모두 Stop한다. 8080, 8081, 8082 등의 포트를 점유하고 있으면 Docker 컨테이너와 충돌한다.

Docker 컨테이너 전체 정리

오전에 Docker Compose로 띄운 컨테이너가 남아있을 수 있다. 전부 정리한다.

```
# 실행 중인 컨테이너 확인
docker ps
```

```
# 모든 컨테이너, 이미지, 볼륨, 네트워크 한 번에 정리
docker system prune -a --volumes
```

y를 입력하면 실행된다. 이 명령은 다음을 모두 삭제한다:

- 모든 컨테이너 (중지된 것 포함)
- 모든 이미지
- 모든 볼륨 (DB 데이터 포함)
- 사용하지 않는 네트워크

포트 사용 확인

정리 후에도 포트가 점유되어 있으면 확인한다.

```
# macOS 기준
lsof -i :8080
lsof -i :8081
lsof -i :8082
lsof -i :8761
```

결과가 나오면 해당 프로세스가 포트를 잡고 있는 것이다. PID를 확인해서 종료한다.

```
kill -9 [PID]
```

아무것도 안 나오면 정상이다.

1-2. AWS 환경 정리

오전에 EC2에 배포한 프로젝트가 있으면 정리한다. SSH로 접속한다.

```
ssh -i [키페어파일].pem ubuntu@[퍼블릭 IP]
```

기존 컨테이너와 이미지 정리

```
# 실행 중인 컨테이너 확인
docker ps

# Docker Compose로 띄운 것이 있으면 해당 디렉토리에서
cd ~/msa-project
docker compose down -v

# 전체 정리
docker system prune -a --volumes
```

기존 프로젝트 디렉토리 정리

```
# 오전에 클론한 프로젝트가 있으면 삭제
rm -rf ~/msa-project
```

디스크 공간 확인

```
df -h
```

20GB 기준으로 사용 가능 용량이 10GB 이상이면 충분하다.

1-3. Day 1 프로젝트 상태 확인

Day 1 실습에서 만든 맛집 리뷰 서비스의 구조를 확인한다.

```
restaurant-review-service/  
├─ member-service/      (회원 서비스 - 8081)  
├─ review-service/      (리뷰 서비스 - 8082)  
└─ api-gateway/         (API Gateway - 8080)
```

review-service에서 member-service를 OpenFeign으로 호출하는 구조이다.

Day 1에서 FeignClient의 url이 하드코딩되어 있는지 확인한다.

```
@FeignClient(name = "member-service", url = "http://localhost:8081")
```

이 `url` 부분을 오늘 Eureka로 대체한다.

1-4. 실습에서 사용할 포트 정리

서비스	로컬 포트	Docker 내부 포트	외부 노출
eureka-server	8761	8761	O (대시보드)
member-service	8081	8081	X
review-service	8082	8082	X
api-gateway	8080	8080	O (API 진입점)
member-db	-	5432	X
review-db	-	5432	X

Docker Compose 환경에서는 Gateway(8080)와 Eureka Dashboard(8761)만 외부에 노출한다. 나머지는 Docker 내부 네트워크에서만 접근 가능하다.

Chapter 2: Eureka 서비스 디스커버리 적용

2-1. eureka-server 프로젝트 생성

Spring Initializr (<https://start.spring.io>)에서 새 프로젝트를 생성한다.

항목	값
Project	Gradle - Groovy
Language	Java
Spring Boot	3.5.10 (안정 버전)
Group	com.example
Artifact	eureka-server
Packaging	Jar
Java	17
Dependencies	Eureka Server

생성한 프로젝트를 `restaurant-review-service/eureka-server/` 경로에 배치한다.

```
restaurant-review-service/  
├─ eureka-server/      ← 새로 추가  
├─ member-service/  
├─ review-service/  
└─ api-gateway/
```

2-2. eureka-server 설정

EurekaServerApplication.java

```
package com.example.eurekaserver;  
  
import org.springframework.boot.SpringApplication;  
import org.springframework.boot.autoconfigure.SpringBootApplication;
```

```
import org.springframework.cloud.netflix.eureka.server.Enable
EurekaServer;

@SpringBootApplication
@EnableEurekaServer
public class EurekaServerApplication {
    public static void main(String[] args) {
        SpringApplication.run(EurekaServerApplication.class,
args);
    }
}
```

application.yml

```
server:
  port: 8761

spring:
  application:
    name: eureka-server

eureka:
  client:
    register-with-eureka: false
    fetch-registry: false
```

eureka-server를 실행하고 <http://localhost:8761>에 접속해서 Eureka Dashboard가 뜨는지 확인한다.

2-3. member-service에 Eureka Client 등록

build.gradle에 의존성 추가

```
ext {
    set('springCloudVersion', "2025.0.1") // Spring Boot 버전
    에 맞게 조정
```

```

}

dependencies {
    // 기존 의존성들...
    implementation 'org.springframework.cloud:spring-cloud-starter-netflix-eureka-client'
}

dependencyManagement {
    imports {
        mavenBom "org.springframework.cloud:spring-cloud-dependencies:${springCloudVersion}"
    }
}

```

Spring Cloud BOM이 이미 있으면 dependencies에 eureka-client만 추가한다.

application.yml에 Eureka 설정 추가

```

# 기존 설정들...

eureka:
  client:
    service-url:
      defaultZone: ${EUREKA_URL:http://localhost:8761/eureka}

```

`${EUREKA_URL:...}` 형태로 환경변수를 지원한다. 로컬에서는 기본값(localhost)을 사용하고, Docker에서는 환경변수로 주입한다.

member-service를 재시작하고 Eureka Dashboard에서 MEMBER-SERVICE가 등록되었는지 확인한다.

2-4. review-service에 Eureka Client 등록

member-service와 동일하게 진행한다.

build.gradle에 의존성 추가 (member-service와 동일)

application.yml에 Eureka 설정 추가 (member-service와 동일)

review-service를 재시작하고 Eureka Dashboard에서 REVIEW-SERVICE가 등록되었는지 확인한다.

2-5. api-gateway에 Eureka Client 등록

Gateway에도 동일하게 Eureka Client를 등록한다.

build.gradle에 의존성 추가 (동일)

application.yml 수정

기존 라우팅에서 고정 URI를 Eureka 서비스명으로 변경한다.

```
spring:
  application:
    name: api-gateway
  cloud:
    gateway:
      routes:
        - id: member-service
          uri: lb://MEMBER-SERVICE          # ← 변경!
          predicates:
            - Path=/api/members/**

        - id: review-service
          uri: lb://REVIEW-SERVICE          # ← 변경!
          predicates:
            - Path=/api/reviews/**

  eureka:
    client:
      service-url:
        defaultZone: ${EUREKA_URL:http://localhost:8761/eureka}
```

기존에 `http://localhost:8081`, `http://localhost:8082` 로 되어있던 부분을 `lb://서비스명` 으로 변경한다.

2-6. FeignClient에서 url 제거

review-service의 FeignClient에서 하드코딩된 url을 제거한다.

```
// Before
@FeignClient(name = "member-service", url = "http://localhost:8081")

// After
@FeignClient(name = "member-service")
```

url을 제거하면 Eureka에서 MEMBER-SERVICE의 실제 주소를 자동으로 찾아간다.

2-7. 동작 확인

4개 서비스를 모두 실행한다. 실행 순서는 다음과 같다:

1. eureka-server (8761)
2. member-service (8081)
3. review-service (8082)
4. api-gateway (8080)

Eureka Dashboard 확인

http://localhost:8761 에서 3개 서비스가 모두 등록되어 있는지 확인한다.

MEMBER-SERVICE	UP (1)
REVIEW-SERVICE	UP (1)
API-GATEWAY	UP (1)

API 테스트

```
# Gateway를 통해 회원 조회
GET http://localhost:8080/api/members/1

# Gateway를 통해 리뷰 조회 (회원 정보 포함)
GET http://localhost:8080/api/reviews/1
```


리뷰 조회 시 회원 이름이 정상적으로 나오면 Eureka를 통한 서비스 디스커버리가 동작하는 것이다.

장애 테스트

member-service를 중지한다. Eureka Dashboard에서 30~90초 후 MEMBER-SERVICE가 사라지는 것을 확인한다. member-service를 다시 시작하면 자동으로 재등록된다.

Chapter 3: Docker Compose 패키징

3-1. application.yml 환경변수 분리

Docker 환경에서는 localhost 대신 컨테이너 이름으로 접근해야 한다. 환경변수를 사용해서 로컬과 Docker 환경을 모두 지원하도록 수정한다.

member-service의 application.yml

```
spring:
  application:
    name: member-service
  datasource:
    url: ${DB_URL:jdbc:postgresql://localhost:5432/member_db}
    username: ${DB_USERNAME:postgres}
    password: ${DB_PASSWORD:password}
  jpa:
    hibernate:
      ddl-auto: update
    show-sql: true

server:
  port: 8081

eureka:
  client:
    service-url:
      defaultZone: ${EUREKA_URL:http://localhost:8761/eureka}
```

review-service의 application.yml

```
spring:
  application:
    name: review-service
  datasource:
    url: ${DB_URL:jdbc:postgresql://localhost:5432/review_db}
    username: ${DB_USERNAME:postgres}
    password: ${DB_PASSWORD:password}
  jpa:
    hibernate:
      ddl-auto: update
    show-sql: true

server:
  port: 8082

eureka:
  client:
    service-url:
      defaultZone: ${EUREKA_URL:http://localhost:8761/eureka}
```

api-gateway의 application.yml

```
spring:
  application:
    name: api-gateway
  cloud:
    gateway:
      routes:
        - id: member-service
          uri: lb://MEMBER-SERVICE
          predicates:
            - Path=/api/members/**

        - id: review-service
```

```

        uri: lb://REVIEW-SERVICE
        predicates:
          - Path=/api/reviews/**

server:
  port: 8080

eureka:
  client:
    service-url:
      defaultZone: ${EUREKA_URL:http://localhost:8761/eureka}

```

`${변수명:기본값}` 형태이므로, 환경변수가 없으면 `:` 뒤의 기본값을 사용한다. 로컬 실행 시에는 기본값(localhost)으로 동작하고, Docker에서는 환경변수를 주입해서 컨테이너 이름으로 접근한다.

3-2. Dockerfile 작성

4개 서비스 모두 동일한 Dockerfile을 사용한다. 각 서비스 디렉토리에 Dockerfile을 만든다.

eureka-server/Dockerfile

```

FROM eclipse-temurin:17-jre
WORKDIR /app
COPY build/libs/*.jar app.jar
ENTRYPOINT ["java", "-jar", "app.jar"]

```

member-service/Dockerfile (동일)

```

FROM eclipse-temurin:17-jre
WORKDIR /app
COPY build/libs/*.jar app.jar
ENTRYPOINT ["java", "-jar", "app.jar"]

```

review-service/Dockerfile (동일)

```
FROM eclipse-temurin:17-jre
WORKDIR /app
COPY build/libs/*.jar app.jar
ENTRYPOINT ["java", "-jar", "app.jar"]
```

api-gateway/Dockerfile (동일)

```
FROM eclipse-temurin:17-jre
WORKDIR /app
COPY build/libs/*.jar app.jar
ENTRYPOINT ["java", "-jar", "app.jar"]
```

3-3. JAR 빌드

Docker 이미지를 만들기 전에 각 서비스의 JAR 파일을 먼저 빌드한다. 프로젝트 루트 디렉토리에서 실행한다.

```
cd eureka-server && ./gradlew clean build -x test && cd ..
cd member-service && ./gradlew clean build -x test && cd ..
cd review-service && ./gradlew clean build -x test && cd ..
cd api-gateway && ./gradlew clean build -x test && cd ..
```

각 서비스의 `build/libs/` 디렉토리에 JAR 파일이 생성되었는지 확인한다.

```
ls eureka-server/build/libs/*.jar
ls member-service/build/libs/*.jar
ls review-service/build/libs/*.jar
ls api-gateway/build/libs/*.jar
```

주의: `-plain.jar` 파일이 함께 생성될 수 있다. Dockerfile에서 `*.jar` 로 복사하면 에러가 난다. 이 경우 각 서비스의 `build.gradle`에 다음을 추가한다.

```
jar {
    enabled = false
}
```

```
}
```

3-4. docker-compose.yml 작성

프로젝트 루트 디렉토리에 docker-compose.yml을 만든다.

```
restaurant-review-service/  
├─ docker-compose.yml      ← 여기에 생성  
├─ eureka-server/  
├─ member-service/  
├─ review-service/  
└─ api-gateway/
```

```
services:
```

```
# 1. Eureka Server
```

```
eureka-server:
```

```
  build: ./eureka-server
```

```
  ports:
```

```
    - "8761:8761"
```

```
  healthcheck:
```

```
    test: ["CMD", "curl", "-f", "http://localhost:8761/actuator/health"]
```

```
    interval: 10s
```

```
    timeout: 5s
```

```
    retries: 5
```

```
# 2. Member DB
```

```
member-db:
```

```
  image: postgres:15
```

```
  environment:
```

```
    POSTGRES_DB: member_db
```

```
    POSTGRES_USER: postgres
```

```
    POSTGRES_PASSWORD: password
```

```

volumes:
  - member-data:/var/lib/postgresql/data

# 3. Member Service
member-service:
  build: ./member-service
  environment:
    DB_URL: jdbc:postgresql://member-db:5432/member_db
    DB_USERNAME: postgres
    DB_PASSWORD: password
    EUREKA_URL: http://eureka-server:8761/eureka
  depends_on:
    eureka-server:
      condition: service_healthy
    member-db:
      condition: service_started

# 4. Review DB
review-db:
  image: postgres:15
  environment:
    POSTGRES_DB: review_db
    POSTGRES_USER: postgres
    POSTGRES_PASSWORD: password
  volumes:
    - review-data:/var/lib/postgresql/data

# 5. Review Service
review-service:
  build: ./review-service
  environment:
    DB_URL: jdbc:postgresql://review-db:5432/review_db
    DB_USERNAME: postgres
    DB_PASSWORD: password
    EUREKA_URL: http://eureka-server:8761/eureka
  depends_on:

```

```

    eureka-server:
      condition: service_healthy
    review-db:
      condition: service_started

# 6. API Gateway
api-gateway:
  build: ./api-gateway
  ports:
    - "8080:8080"
  environment:
    EUREKA_URL: http://eureka-server:8761/eureka
  depends_on:
    eureka-server:
      condition: service_healthy

volumes:
  member-data:
  review-data:

```

외부에 노출하는 포트는 Gateway(8080)와 Eureka Dashboard(8761) 두 개뿐이다.
member-service, review-service, DB는 Docker 내부 네트워크에서만 접근 가능하다.

3-5. 로컬에서 Docker Compose 실행

IntelliJ에서 실행 중인 서비스가 있으면 모두 중지한다.

```
docker compose up --build -d
```

컨테이너 상태를 확인한다.

```
docker compose ps
```

6개 컨테이너가 모두 Running 또는 Healthy 상태인지 확인한다.

NAME	STATUS
eureka-server-1	Up (healthy)
member-db-1	Up
member-service-1	Up
review-db-1	Up
review-service-1	Up
api-gateway-1	Up

서비스가 시작되지 않았으면 로그를 확인한다.

```
# 특정 서비스 로그 확인
docker compose logs member-service
docker compose logs review-service

# 전체 로그를 실시간으로 확인
docker compose logs -f
```

3-6. 동작 확인

Eureka Dashboard

<http://localhost:8761> 에서 3개 서비스가 등록되어 있는지 확인한다.

API 테스트

```
# 회원 등록
POST http://localhost:8080/api/members
Body: {"name": "홍길동", "email": "hong@test.com"}

# 회원 조회
GET http://localhost:8080/api/members/1

# 리뷰 작성
POST http://localhost:8080/api/reviews
Body: {"memberId": 1, "restaurantName": "맛있는 김치찌개", "content": "최고!", "rating": 5}
```



```
# 리뷰 조회 (회원 이름 포함)
GET http://localhost:8080/api/reviews/1
```

리뷰 조회 시 회원 이름이 함께 나오면 Docker Compose 환경에서 서비스 간 통신이 정상 동작하는 것이다.

3-7. 유용한 Docker Compose 명령어

```
# 전체 중지 (컨테이너 유지)
docker compose stop

# 전체 중지 + 컨테이너 삭제 (볼륨 유지)
docker compose down

# 전체 중지 + 컨테이너 + 볼륨 삭제 (DB 데이터도 삭제)
docker compose down -v

# 특정 서비스 로그 확인
docker compose logs -f review-service

# 특정 서비스만 재시작
docker compose restart member-service

# 코드 수정 후 특정 서비스만 재빌드
docker compose up --build -d member-service
```

Chapter 4: AWS EC2 배포

4-1. EC2 인스턴스 준비

Day 1에서 사용한 인스턴스가 있으면 그대로 사용한다. 없으면 새로 생성한다.

새로 생성하는 경우

항목	값
이름	student[번호]-server
AMI	Ubuntu 24.04 LTS
인스턴스 유형	t3.medium (4GB RAM)
스토리지	20GB
키 페어	기존 키 페어 사용 또는 새로 생성

보안 그룹 인바운드 규칙

포트	용도
22	SSH 접속
8080	API Gateway
8761	Eureka Dashboard

t3.medium을 선택하는 이유는 JVM 서비스 4개 + PostgreSQL 2개를 동시에 실행하면 약 2GB 이상의 메모리가 필요하기 때문이다. t3.small(2GB)은 OOM 위험이 있다.

4-2. SSH 접속 및 기존 환경 정리

```
ssh -i [키페어파일].pem ubuntu@[퍼블릭 IP]
```

오전에 배포한 프로젝트가 남아있을 수 있다. 정리한다.

```
# 실행 중인 컨테이너 확인
docker ps

# Docker Compose로 띄운 것이 있으면
cd ~/msa-project
docker compose down -v

# 전체 정리
docker system prune -a --volumes -f

# 기존 프로젝트 삭제
```

```
cd ~  
rm -rf msa-project
```

Docker가 아직 설치되지 않은 경우 (설치 안 했으면) 설치한다.

```
# 시스템 업데이트  
sudo apt update -y  
  
# Docker 설치  
sudo apt install -y docker.io  
sudo systemctl start docker  
sudo systemctl enable docker  
sudo usermod -aG docker ubuntu  
  
# Docker Compose 설치  
sudo apt install -y docker-compose-v2  
  
# Java 17 설치  
sudo apt install -y openjdk-17-jdk  
  
# 적용을 위해 SSH 재접속  
exit
```

```
ssh -i [키페어파일].pem ubuntu@[퍼블릭 IP]
```

재접속 후 docker 명령이 sudo 없이 동작하는지 확인한다.

```
docker ps  
docker compose version  
java -version
```

4-3. 프로젝트 업로드

GitHub에 프로젝트를 push한 경우:

```
cd ~
git clone https://github.com/[본인계정]/restaurant-review-service.git
cd restaurant-review-service
```

GitHub를 사용하지 않는 경우, 로컬에서 scp로 전송한다. 로컬 터미널에서 실행한다:

```
scp -i [키페어파일].pem -r ./restaurant-review-service ubuntu@[퍼블릭 IP]:~/
```

4-4. JAR 빌드

EC2에서 각 서비스의 JAR 파일을 빌드한다.

```
cd ~/restaurant-review-service

cd eureka-server && ./gradlew clean build -x test && cd ..
cd member-service && ./gradlew clean build -x test && cd ..
cd review-service && ./gradlew clean build -x test && cd ..
cd api-gateway && ./gradlew clean build -x test && cd ..
```

첫 빌드 시 Gradle이 의존성을 다운로드하므로 시간이 걸린다. 2~3분 정도 기다린다.

빌드가 완료되면 JAR 파일이 생성되었는지 확인한다.

```
ls eureka-server/build/libs/*.jar
ls member-service/build/libs/*.jar
ls review-service/build/libs/*.jar
ls api-gateway/build/libs/*.jar
```

4-5. Docker Compose로 배포

```
docker compose up --build -d
```

컨테이너 상태를 확인한다.

```
docker compose ps
```

6개 컨테이너가 모두 올라오는 데 1~2분 정도 걸린다. eureka-server의 healthcheck가 통과한 뒤에 나머지 서비스가 시작된다.

서비스가 올라오지 않으면 로그를 확인한다.

```
docker compose logs eureka-server
docker compose logs member-service
docker compose logs review-service
```

4-6. 동작 확인

Eureka Dashboard

브라우저에서 `http://[퍼블릭 IP]:8761` 에 접속한다. 3개 서비스가 등록되어 있는지 확인한다.

API 테스트

회원 등록

POST `http://[퍼블릭 IP]:8080/api/members`

Body: `{"name": "홍길동", "email": "hong@test.com"}`

회원 조회

GET `http://[퍼블릭 IP]:8080/api/members/1`

리뷰 작성

POST `http://[퍼블릭 IP]:8080/api/reviews`

Body: `{"memberId": 1, "restaurantName": "맛있는 김치찌개", "content": "최고!", "rating": 5}`

리뷰 조회

GET `http://[퍼블릭 IP]:8080/api/reviews/1`

휴대폰에서 또는 브라우저에서 접속 확인

휴대폰 브라우저에서 `http://[퍼블릭 IP]:8080/api/members/1` 에 접속해본다. 응답이 나오면 AWS 배포가 완료된 것이다.

4-7. 트러블슈팅

접속이 안 되는 경우

보안 그룹에서 8080, 8761 포트가 열려있는지 확인한다. 소스를 `0.0.0.0/0` 으로 설정해야 외부에서 접근 가능하다.

서비스가 계속 재시작하는 경우

메모리 부족일 수 있다. 확인한다.

```
free -h
docker stats
```

Available 메모리가 500MB 미만이면 t3.medium으로 인스턴스 유형을 변경해야 한다.

빌드 중 디스크 부족

```
df -h
```

사용 가능 용량이 2GB 미만이면 Docker 캐시를 정리한다.

```
docker system prune -a -f
```

Chapter 5: 스케일 아웃 & Circuit Breaker

5-1. review-service 스케일 아웃

Docker Compose에서 review-service를 2개로 늘린다.

```
docker compose up --scale review-service=2 -d
```

컨테이너 상태를 확인한다.

```
docker compose ps
```

review-service가 2개 실행되고 있는지 확인한다.

NAME	STATUS
review-service-1	Up
review-service-2	Up

Eureka Dashboard(<http://localhost:8761>)에서 REVIEW-SERVICE의 인스턴스가 2개로 표시되는지 확인한다. 등록까지 30초 정도 걸릴 수 있다.

5-2. 로드 밸런싱 확인

요청이 2개의 review-service에 분산되는지 확인하기 위해 로그를 확인하는 방법을 사용한다.

review-service의 Controller에 로그를 추가한다. 이미 추가되어 있으면 건너뛰다.

```
@Slf4j
@RestController
@RequestMapping("/api/reviews")
public class ReviewController {

    @GetMapping("/{id}")
    public ResponseEntity<ReviewResponseDto> getReview(@PathVariable Long id) {
        log.info("리뷰 조회 요청 - port: {}", System.getenv("SERVER_PORT"));
        // ...
    }
}
```

Docker 환경에서는 `SERVER_PORT` 대신 호스트명으로 구분하는 것이 더 명확하다. 다음과 같이 변경한다.

```
import java.net.InetAddress;
```

```

@GetMapping("/{id}")
public ResponseEntity<ReviewResponseDto> getReview(@PathVariable Long id) {
    try {
        log.info("리뷰 조회 요청 - hostname: {}", InetAddress.getLocalHost().getHostName());
    } catch (Exception e) {
        log.info("리뷰 조회 요청");
    }
    // ...
}

```

코드를 수정했으면 다시 빌드하고 배포한다.

```

cd review-service && ./gradlew clean build -x test && cd ..
docker compose up --build --scale review-service=2 -d

```

같은 API를 여러 번 호출한다.

```

GET http://localhost:8080/api/reviews/1
GET http://localhost:8080/api/reviews/1
GET http://localhost:8080/api/reviews/1
GET http://localhost:8080/api/reviews/1

```

로그에서 hostname이 번갈아 나오는지 확인한다.

```

docker compose logs -f review-service

```

```

review-service-1 | 리뷰 조회 요청 - hostname: abc123def
review-service-2 | 리뷰 조회 요청 - hostname: fed321cba
review-service-1 | 리뷰 조회 요청 - hostname: abc123def
review-service-2 | 리뷰 조회 요청 - hostname: fed321cba

```

두 개의 hostname이 번갈아 나오면 로드 밸런싱이 동작하고 있는 것이다.

5-3. Circuit Breaker 의존성 추가

review-service에서 member-service를 호출할 때 Circuit Breaker를 적용한다.
member-service가 죽더라도 review-service가 함께 죽지 않도록 보호하는 것이다.

review-service의 build.gradle에 의존성 추가

```
dependencies {  
    // 기존 의존성들...  
    implementation 'org.springframework.cloud:spring-cloud-starter-circuitbreaker-resilience4j'  
}
```

review-service의 application.yml에 Circuit Breaker 설정 추가

기존 설정 아래에 추가

```
resilience4j:  
  circuitbreaker:  
    instances:  
      member-service:  
        sliding-window-size: 5  
        failure-rate-threshold: 50  
        wait-duration-in-open-state: 30s  
        permitted-number-of-calls-in-half-open-state: 3
```

설정	의미
sliding-window-size: 5	최근 5번의 호출을 기준으로 판단
failure-rate-threshold: 50	실패율이 50% 이상이면 회로 열림
wait-duration-in-open-state: 30s	회로가 열리면 30초간 호출 차단
permitted-number-of-calls-in-half-open-state: 3	30초 후 3번 시도해서 정상이면 회로 닫힘

5-4. MemberClientWrapper 클래스 생성

`@CircuitBreaker`를 FeignClient에 직접 붙이지 않고 Wrapper 클래스를 만든다. Spring AOP는 같은 클래스 내부 호출에서는 프록시를 거치지 않기 때문에 별도 클래스로 분리해야 어노테

이션이 동작한다.

MemberClientWrapper.java

```
package com.example.reviewservice.client;

import io.github.resilience4j.circuitbreaker.annotation.CircuitBreaker;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.springframework.stereotype.Component;

@Slf4j
@Component
@RequiredArgsConstructor
public class MemberClientWrapper {

    private final MemberClient memberClient;

    @CircuitBreaker(name = "member-service", fallbackMethod = "getMemberFallback")
    public MemberDto getMember(Long memberId) {
        return memberClient.getMember(memberId);
    }

    public MemberDto getMemberFallback(Long memberId, Exception e) {
        log.warn("member-service 호출 실패! fallback 실행. memberId={}, error={}",
            memberId, e.getMessage());
        return new MemberDto(memberId, "알 수 없는 사용자", null);
    }
}
```

fallback 메서드의 규칙:

- 리턴 타입은 원본 메서드와 동일해야 한다.
- 파라미터는 원본 파라미터 + 마지막에 Exception을 추가한다.
- MemberDto 생성자의 필드 개수를 맞춰야 한다. 본인의 MemberDto 구조에 맞게 조정한다.

5-5. ReviewService 수정

기존에 MemberClient를 직접 사용하던 부분을 MemberClientWrapper로 변경한다.

```
@Service
@RequiredArgsConstructor
public class ReviewService {

    private final ReviewRepository reviewRepository;
    // private final MemberClient memberClient;          //
제거
    private final MemberClientWrapper memberClientWrapper; //
변경

    public ReviewResponseDto getReview(Long id) {
        Review review = reviewRepository.findById(id)
            .orElseThrow(() -> new RuntimeException("리뷰
를 찾을 수 없습니다."));

        // MemberDto member = memberClient.getMember(review.g
etMemberId());    // 제거
        MemberDto member = memberClientWrapper.getMember(revi
ew.getMemberId()); // 변경

        return ReviewResponseDto.from(review, member);
    }

    // 다른 메서드에서 memberClient를 사용하는 부분도 동일하게 변경한
```

```
다.  
}
```

5-6. 빌드 및 배포

```
cd review-service && ./gradlew clean build -x test && cd ..  
docker compose up --build --scale review-service=2 -d
```

정상 동작을 먼저 확인한다.

```
GET http://localhost:8080/api/reviews/1
```

리뷰 조회 시 회원 이름이 정상적으로 나오는지 확인한다.

5-7. 장애 테스트

member-service를 강제로 중지한다.

```
docker compose stop member-service
```

리뷰를 다시 조회한다.

```
GET http://localhost:8080/api/reviews/1
```

응답에서 회원 이름이 "알 수 없는 사용자"로 나오면 Circuit Breaker의 fallback이 동작하는 것이다.

```
{  
  "id": 1,  
  "restaurantName": "맛있는 김치찌개",  
  "content": "최고!",  
  "rating": 5,  
  "memberName": "알 수 없는 사용자"  
}
```

review-service의 로그를 확인한다.

```
docker compose logs -f review-service
```

```
WARN - member-service 호출 실패! fallback 실행. memberId=1, error=...
```

Circuit Breaker 상태 변화 확인

같은 요청을 빠르게 5번 이상 보낸다. sliding-window-size가 5이고 실패율이 100%이므로 회로가 열린다. 회로가 열리면 member-service에 요청을 보내지 않고 바로 fallback을 실행한다. 로그에서 에러 메시지가 `CircuitBreaker 'member-service' is OPEN` 으로 바뀌는 것을 확인할 수 있다.

복구 확인

member-service를 다시 시작한다.

```
docker compose start member-service
```

30초(wait-duration-in-open-state) 후에 리뷰를 다시 조회한다. 회원 이름이 다시 정상적으로 나오면 Circuit Breaker가 HALF_OPEN → CLOSED 상태로 복구된 것이다.

5-8. Circuit Breaker 동작 정리

정상 상태 (CLOSED)

- member-service 호출 성공
- 회원 이름 정상 반환

member-service 장애 발생

- 최근 5번 중 50% 이상 실패
- 회로 열림 (OPEN)
- 더 이상 호출하지 않고 바로 fallback 실행
- "알 수 없는 사용자" 반환

30초 경과

- 반열림 (HALF_OPEN)

→ 3번 시도해봄

member-service 복구됨

→ 3번 모두 성공

→ 회로 닫힘 (CLOSED)

→ 정상 동작 재개

Circuit Breaker가 없으면 member-service 장애 시 review-service도 같이 응답 불가 상태가 된다. Circuit Breaker가 있으면 완벽한 데이터는 아니지만 서비스 자체는 계속 응답할 수 있다.